

HW 5: Centrality & Games

Assigned: 02/13/2026

Due: 02/19/2026 by 11:59 PM PT

Collaboration is allowed for all problems and at the top of your homework sheet, please list all the people with whom you discussed. Crediting help from other classmates will not take away any credit from you. The details of the collaboration policy for this course are available in the Resources tab on Piazza.

*You must turn in your homework electronically via Gradescope. **Be sure to submit your homework as a single file.***

Start early and come to office hours with your questions! We also encourage you to post your questions on Piazza, as well as answer the questions asked by others on Piazza.

Coding + Data Analysis [70 points]

1 Discovering Clusters [40 points]

In class, you have seen the power of clustering on many real world applications. The choice of clustering algorithm depends on both the structures of graphs and the metrics used for clustering, among other things. In this problem, you will evaluate various clustering algorithms on a couple of different types of graphs: symmetric stochastic block model (SSBM), as seen in **Homework 2**, and the "circle graph", to get a sense for the differences between algorithms.

Your task:

- (a) **[20 points]** Generate a SSBM network with $n = 30, k = 3, A = 0.7$ and $B = 0.1$. Run the spectral clustering and k -means clustering algorithms over the network, for appropriate value(s) of k , and visualize the clusters obtained from each algorithm. You may find Python libraries such as `sklearn.cluster.KMeans` and `sklearn.cluster.SpectralClustering` useful for this question.

Reminder

The *stochastic block model (SBM)* is a random graph model with planted clusters. The model $SBM(n, p, W)$ defines an n -vertex random graph with labeled vertices for positive integers n , k , a probability vector p of dimension k , and a symmetric matrix W of dimension $k \times k$ with entries $W_{i,j} \in [0, 1]$. Each vertex is assigned a community label in $\{1, \dots, k\}$ independently under the community prior p , and pairs of vertices with community labels i and j connect independently with probability $W_{i,j}$. The SBM is called *symmetric* if p is uniform and if W takes the same value A on the diagonal and the same value B outside the diagonal. We then represent the symmetric SBM as $SSBM(n, k, A, B)$.

Turn in a visualization of the results of your algorithms and a discussion of any differences you notice between the two algorithms. You may base this explanation on the visualization or some

specific clustering metrics of your choice. Also, discuss how well the clusters were recovered by these algorithms by comparing them to the actual clusters that were used to form the SSBM instance you generated.

- (b) **[20 points]** Generate an instance of the "circle graph", as seen below in Figure 1, with number of nodes $n = 500$, and noise $\sigma = 0.01$, using the `sklearn.datasets.make_circles` function; half of the n nodes will end up in the inner circle, and the other half in the outer circle. Run the spectral clustering and k -means clustering algorithms over the network, for appropriate value(s) of k , and visualize the clusters obtained from each algorithm. Discuss any differences you notice between the two algorithms, and why any differences might occur. You will again find the aforementioned Python libraries useful for this problem.

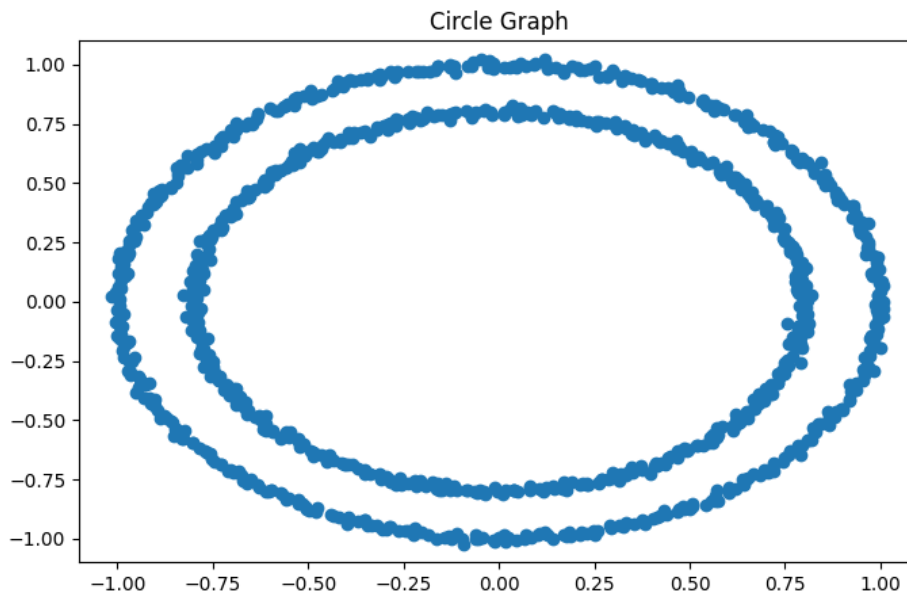


Figure 1: The Circle Graph with $n = 500, \sigma = 0.01$

Note 1

The two clustering algorithms differ in that spectral clustering conventionally operates on a graph $G = (V, E)$ represented using an adjacency matrix (which is how the SSBM graph is represented), whereas k -means conventionally operates on a graph whose nodes are points in a vector space (which is how the circle graph is represented: a set of (x, y) coordinate pairs). Therefore, in order to run spectral clustering on the circle graph, in addition to passing in the matrix of data points, you will need to pass in an "affinity" function, which essentially defines a metric by which the algorithm can build an adjacency graph from the points in the vector space; we suggest the `nearest_neighbors` affinity function, which creates an adjacency matrix with edges between nodes that are close to each other in the vector space. When running k -means on the SSBM graph, you can simply pass in the adjacency matrix as is, or transform it as you see fit; regardless of which one you choose, as part of your discussion, talk about how the k -means algorithm is interpreting the data points you pass in to generate its clustering. Running k -means on the circle graph and running spectral clustering on the SSBM graph should follow naturally from the conventional definitions of the two algorithms.

Note 2

For those of you interested in using a language/library besides Python/sklearn, the circle graph is created by first generating a vector of equally spaced points $\{r_1, \dots, r_{n/2}\}$ in the interval $[0, 2\pi)$, then generates (x, y) coordinate pairs according to $x_i^{\text{outer}} = (\cos(r_i), \sin(r_i))$ for the outer circle and $x_i^{\text{inner}} = f \cdot (\cos(r_i), \sin(r_i))$ for the inner circle, where $0 \leq f < 1$ (the `make_circles` function uses $f = 0.8$ by default.) Then, a Gaussian error term $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ is added to the x coordinate of each of the n points, to give the final dataset.

2 Braess's Paradox Simulation [30 points]

In game theory and traffic assignment, *Braess's Paradox* is the counter-intuitive observation that adding road capacity to a network can sometimes *increase* the overall travel time for everyone. This happens because individual drivers act selfishly to minimize their own travel time, reaching a Nash Equilibrium that is not necessarily the social optimum. The new road might offer a shortcut that looks attractive to individuals, but when everyone tries to use it, the resulting congestion slows the whole system down. In this problem, you will simulate this phenomenon to see how selfish routing decisions lead to suboptimal outcomes.

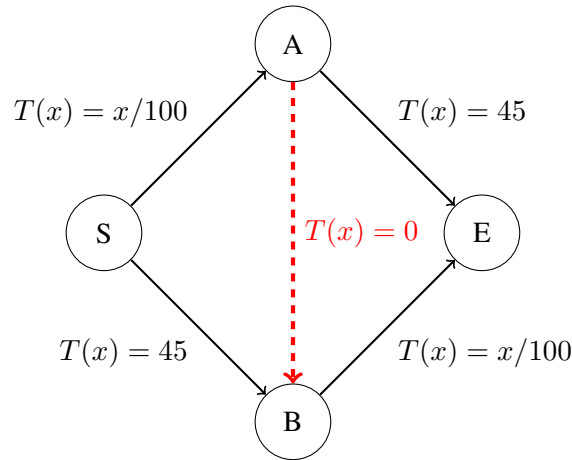


Figure 2: The Traffic Network. Solid lines are the initial state; the dashed line is the upgrade.

Your task: Simulate 4000 drivers traveling from S to E .

- [10 points]** Initialize all drivers on the top ($S \rightarrow A \rightarrow E$) path. Iteratively move 1% of the drivers currently on the slowest path to the fastest path; do this 1,000 times. Assume drivers act greedily, are rational, and make their decisions in each iteration simultaneously. Generate a plot showing average travel time for each path and overall v. iteration number. Report the final number of drivers on each path and the average travel time. Does this appear to be an equilibrium?
- [10 points]** Add the zero-cost edge $A \rightarrow B$, creating a third path ($S \rightarrow A \rightarrow B \rightarrow E$). Continuing from the final distribution of drivers between the top and bottom paths from the previous part, now allow drivers to switch to any of the three paths. Generate a plot showing average travel time for each path and overall v. iteration number. Report the new average travel time and the final number of drivers on each path. Does this appear to be an equilibrium? Did the average travel time increase or decrease?
- [10 points]** Explain intuitively why rational drivers flocked to the new path despite it worsening the global outcome. Why don't they switch back?

Theory [30 Points]

3 Eavesdropping Centrality [15 points]

For this problem, you'll study a new notion of centrality. The idea behind the measure is that the "importance" of a node in a communication network is, in some sense, tied to the desire of an eavesdropper to monitor network communication by "bugging" that node. If the node would be really important for an eavesdropper to bug then the node is "central" in some sense.

Concretely, consider the following setting.

- **Communication Process:** Between time 0 and time 1, each vertex sends a (different) message to each of its neighbors independently at a uniformly random time, i.e., for each edge (i, j) , T_{ij} is the time i sends a message to j and is distributed between $[0, 1)$ independently of all other edges.
- **Eavesdropping Process:** At any time between time 0 and time 1 the eavesdropper can "bug" any set of nodes in the network and intercept the incoming messages. But, the probability the eavesdropper is caught by node i at the end of the round is t_i^β where t_i is the total time spent at node i during the round and β is a parameter. The events of being caught at node i and node j are not independent.

The policy of the eavesdropper is $t = (t_1, \dots, t_n)$, which specifies how much time each node is monitored and the goal of the eavesdropper is to maximize the expected number of messages intercepted during the round while making sure that the probability of being caught is smaller than γ , where $\gamma \in (0, 1)$.

Intuitively, the importance of a node is proportional to the amount of time it is bugged by a strategic eavesdropper.

- (a) [5 points] Given policy $t = (t_1, \dots, t_n)$, show that the expected number of intercepted messages is $\sum_{i=1}^n t_i d_i$ where d_i is the degree of node i .
- (b) [5 points] Prove that if $\sum_{i=1}^n t_i^\beta \leq \gamma$, then the probability of being caught is at most γ . This means that such a $t = (t_1, \dots, t_n)$ is a valid policy.
- (c) [5 points] Given **parts (a) and (b)**, we obtain an optimization problem that can be solved to determine the vector of eavesdropper centrality, t^* :

$$t^* = \arg \max \sum_{i=1}^n t_i d_i \text{ subject to } \sum_{i=1}^n t_i^\beta \leq \gamma. \quad (1)$$

Note that this problem uses a bound on the probability of being caught. We'll ignore that fact for this homework, but please convince yourself why that's okay! Our goal in this problem is to connect the idea of eavesdropping centrality to the notions of centrality in the previous problem.

To that end, consider two specific cases: $\beta = 1$ and $\beta = 2$. For each of these cases, solve the optimization problem and interpret the resulting t^* as a centrality measure, connecting it to the centrality measures in the previous problem. (*Hint: The Cauchy-Schwarz inequality may be useful.*)

4 Game Theory Warm Up [10 points]

This question is designed to put in use the concepts you learned in the **game theory refresher lecture**. Remember that for each element in the table, (x, y) represents a payoff of x for Player 1 and y for Player 2.

		Player 2			
		a	b	c	d
Player 1	w	(3, 2)	(4, 1)	(4, 0)	(2, 1)
	x	(2, 4)	(5, 3)	(5, 5)	(0, 3)
	y	(2, 1)	(3, 3)	(3, 0)	(0, 6)
	z	(1, 3)	(4, 6)	(5, 4)	(3, 1)

Consider the two-player normal form game given above.

- [5 points] Which actions survive the iterated elimination of dominated actions? Explain your claim.
- [5 points] Find all Nash Equilibria (*Hint: you should find 2 pure and 2 mixed*).

5 Routing Games with Tolls [5 points]

In our discussion of routing games in class, we talked about using tolls to induce independent selfish players to behave in a manner that is optimal for the system. To this end, we discussed the principle of *marginal cost pricing*. In this exercise, you'll prove that marginal cost pricing really works.

Consider the model of routing games described in class. Assume that each user i has traffic $r_i = 1$. We add a 'toll' to the original link cost functions $c_e(\cdot)$, setting the new cost function to be

$$c_e^*(x) = c_e(x) + (x - 1)(c_e(x) - c_e(x - 1))$$

Notice that the 'toll' on link e captures the net latency that each player using the link adds to the other players using it.

Your Task: Prove that any optimal strategy for our original game is a Nash equilibrium for this new game (with link cost functions $c_e^*(\cdot)$). Recall that the objective function in the original game is $\sum_{i \in N} \sum_{e \in E_i} c_e(f_e) = \sum_{e \in E} f_e c_e(f_e)$ where f_e is the flow on e , N is the set of players and E_i contains the edges on the path selected by player i . (*Hint: Think about constructing a new potential function with the new cost.*)